大数据招聘岗位数据分析

以 29,999 条招聘岗位 JD 为样本，回答一个具体问题：一个准备投大数据方向实习的学生，应该把有限的时间花在哪几项技能上。

PROJECT	BACKGROUND	COMPLETED	STACK
个人独立项目	大数据技术 · **专科大三**	2026 年 10 月	Python · pandas · matplotlib

项目从一个很实际的问题出发：**简历上的技能栏，该写哪几行、写几行。** 我把 3 万条招聘信息从「一段段读不懂的岗位描述」，变成「一张能比较、能排序、能下判断的表」， 并用四张图给出答案；也用一个「负结果」说明：数据里有些字段是不能用来下结论的。

1 项目概述

1.1 为什么做这个项目

大三上学期，身边的同学开始分头准备：有人报培训班学 Java，有人在刷 SQL 题，有人去学推荐算法。 我问自己一个问题——**这些技能里，哪些是招聘方真正写在要求里的，哪些只是大家互相传的。** 凭印象回答不靠谱：我打开招聘 App，看到的是一条条独立的岗位描述，读完几十条，脑子里只有「好像都要 SQL」，说不出比例，也排不出先后。

于是我决定把这件事做成数据问题：把大量岗位描述抓下来，把「要不要某项技能」变成可以统计的 0/1 字段， 用覆盖率、分布、共现关系去回答。这份报告就是整个过程：从采集口径，到清洗规则，到四张图和四条结论， 也包括我踩过的坑和没能解决的问题。

1.2 要回答的问题

项目围绕三个业务问题展开，第四个问题是分析过程中才意识到更重要的：

- **Q1 学什么：**各项技能在招聘要求里出现的比例是多少？哪一项是真门槛，哪一项只是加分项？
- **Q2 学多宽：**一份岗位平均要求几项技能？继续往上堆技能，还有没有额外收益？
- **Q3 怎么组合：**技能之间是成套出现的，还是彼此独立？不同岗位类别要不要不同的技能栈？
- **Q4 可信吗：**我准备写进结论的字段，本身有没有区分度？（这一问后来直接决定了一张图不进结论。）

1.3 我的工作与产出

项目由我独立完成，覆盖「口径设计 → 数据清洗 → 探索分析 → 可视化 → 结论输出」全流程， 主要使用 Python 的 pandas 做数据处理、matplotlib 做图表，全部步骤脚本化，可复跑。

产出物	说明
结构化数据集	29,999 行 × 27 列，含 20 个派生字段（薪资结构化、学历/经验编码、13 个技能标签），零缺失

产出物	说明
可视化图表	4 张，覆盖单变量分布、分布形态、双变量结构与字段可信度检验
可复跑脚本	`recruit_viz.py`，一条命令输出 4 张图，替换 CSV 路径即可复用到其他招聘数据
分析报告	本页（网页版）与 A4 PDF（用于随简历附交）

1.4 主要结果速览

结论	关键数字	证据强度
SQL 是唯一的高频硬技能	58.7%	**强（n=29,999，单变量统计）**
单岗技能要求中位数 4 项，93% 不超过 5 项	4 项 / 93.0%	**强（分布形态清晰）**
SQL 与 MySQL 是唯一强绑定的一组技能	100% / 60%	**强（条件概率，提升 1.7 倍）**
薪资字段无区分度，不能用于择业与谈薪	r = -0.006	**负结果（已排除出结论）**

表中「证据强度」是我对每条结论的可信程度做的自评：单变量与分布类结论依赖样本量，条件概率依赖共现结构，负结果依赖相关性检验。

---

2 数据源与采集方案

2.1 数据源的选择

做「市场上要什么技能」这类问题，有三种常见数据来源，我逐项比较后选了第一种：

候选来源	优点	问题	是否采用
招聘平台公开岗位页	数量大、更新快；薪资/城市/学历/经验都有结构化标签；JD 正文即需求原文	只代表「在招岗位」，不代表在职人群；需遵守平台规则	**采用**
公开统计数据集	已清洗、合规	字段粗，通常只有薪资均值，看不到具体技能要求	不采用
问卷 / 访谈	能问到真实工作内容	样本小、主观、成本高，不适合量化技能比例	作为后续补充

选择招聘平台的核心原因是**「需求侧原文」**：岗位描述里写的技能，就是企业愿意为之付钱的能力清单， 比任何二手总结都更接近事实。

2.2 采集范围设计

采集范围决定了结论的适用范围，所以先定边界：城市、岗位关键词、行业三个维度组合检索，控制在同一时间窗口内， 避免不同月份招聘旺季差异带来的干扰。

维度	取值	设定理由
城市（12）	北京、上海、深圳、广州 成都、杭州、武汉、南京、西安、重庆、长沙、郑州	4 个一线 + 8 个新一线，覆盖大数据岗位的主要就业地，也便于比较城市层级差异

维度	取值	设定理由
岗位关键词（5）	数据分析工程师、大数据开发工程师、大数据运维、数据运营、数据专员	从「偏分析」到「偏工程」再到「偏业务支持」的完整光谱，能检验不同岗位类别是否要求不同技能
行业（7）	互联网、电商、金融、通信、制造业、医疗、政务	覆盖数字化投入较高与正在转型的两类行业
时间窗口	2026 年 9 月，连续约两周	同一窗口内横向比较才成立；跨月比较会混入招聘节奏差异

2.3 采集字段

原始采集 7 个文本字段，其余 20 个字段由清洗阶段生成（详见第 3 节），最终形成 27 列的分析表。

字段	类型	示例	用途
岗位名称	文本	数据分析工程师	岗位类别分组
薪资区间	文本	21k-29k	待结构化为数值
工作城市	文本	成都	城市维度、城市分级
学历要求	文本	本科	编码为 1/2/3
工作经验	文本	应届生 / 3-5年	编码为年限数值
岗位描述	文本	需要掌握 Python、数据仓库……	技能标签抽取的原文依据
所属行业	文本	制造业	行业维度交叉
salary_min / max / avg	数值	21.0 / 29.0 / 25.0	薪资分析（最终未采用，见 4.4）
education_level / exp_year	整数	2 / 5	便于排序与相关性检验
city_tier	文本	新一线城市	城市层级对比
desc_len	整数	45	描述详细度
skill_*（13 个）	0/1	skill_SQL = 1	技能覆盖率、共现分析的核心字段

2.4 采集与入库流程

流程共五步，每一步都做了可核对的设计，避免「抓完才发现口径不一致」：

1. **组合任务：**5 个岗位关键词 × 12 个城市 = 60 个检索任务，逐个执行。
2. **列表页翻取：**按发布时间排序翻页，记录岗位链接与列表页摘要，先落库再进详情页，避免中途失败全部白跑。
3. **详情页解析：**只取 2.3 节列出的 7 个字段，其余内容不保存，减少无关信息与合规风险。
4. **去重入库：**按岗位链接与「岗位名称 + 城市 + 薪资 + 描述」指纹去重，同一岗位多城市发布只保留一条。
5. **抽样核对：**随机抽 200 条人工比对原文，检查字段是否错位、薪资是否解析错误，修正后再进入清洗。

2.5 合规与伦理

- 只采集**公开可见**的岗位信息，不采集 HR 姓名、联系方式等个人信息。
- 请求之间限速 1–2 秒，遵守目标站点的 robots 协议与服务条款，仅用于课程与求职作品用途。
- 原始数据**不二次分发**；报告中只呈现聚合统计结果，不出现具体公司名称与岗位原文。

2.6 样本规模与一个必须提前说明的特征

清洗后有 **29,999 条**有效记录。需要提前说明的是：各城市样本量都在 2,434–2,562 条之间， 各岗位在 5,875–6,074 条之间，五个岗位几乎平均分布——这说明本次采集是**按城市与岗位做了配额抽样**， 而不是自然分布。

好处是每个分组的样本量足够，分组比较的误差小；代价是**不能用它判断「哪个城市岗位更多」「哪类岗位更缺人」**。 这条限制在第 6 节会再次出现，它约束了本项目能说的结论边界。

---

3 数据预处理与清洗

3.1 原始数据有五类问题

刚抓下来的数据不能直接分析，问题集中在「人和机器对同一句话的理解不一致」上：

问题	原始样子	不处理会怎样
薪资是文本	21k-29k、1.5-2万、面议	无法求均值、无法画箱线图
分类变量是中文	专科 / 本科 / 硕士；应届生 / 3-5年	无法排序，也做不了相关性检验
技能藏在长文本里	「需要掌握 Python、数据仓库，负责数据处理……」	无法统计覆盖率，只能靠人读
城市没有层级	成都、西安、郑州	无法回答「一线和新一线差多少」
存在重复与缺失	同一岗位多城市发布；个别记录无薪资	统计结果被重复样本放大

3.2 清洗流程：七个步骤

每一步都记录「目的 → 做法 → 产出字段」，全部写成函数，保证换数据也能重跑。

步骤	目的	做法
S1 去重	避免同一岗位被重复计数	岗位名称 + 城市 + 薪资 + 描述拼接后取 MD5，重复只保留首条
S2 必填校验	保证分析字段完整	薪资、城市、学历、经验任一为空即剔除；后续校验缺失值为 0
S3 薪资结构化	把文本变成可计算的数值	正则提取上下限，统一换算为「k/月」（年薪 ÷ 12、元/天 × 21.75），生成 `salary_min`、`salary_max`、`salary_avg` =（下限 + 上限）÷ 2；「面议」剔除
S4 分类编码	让分类变量可排序、可检验	学历：专科 1 / 本科 2 / 硕士 3；经验：应届 0 / 1-3年 3 / 3-5年 5 / 5-10年 10（取区间上界）；城市按一线 / 新一线打标
S5 技能抽取	把描述文本变成 0/1 标签	建 13 项技能词典（含同义词，如「关系型数据库 / MySQL / Mysql」都归到 MySQL），在描述上做不区分大小写的正则匹配；同一技能出现多次只记 1
S6 派生字段	为后续分析准备维度	`desc_len` 描述字数；薪资区间宽度 = 上限 − 下限
S7 异常值	防止极端值扭曲图形	下限 > 上限剔除；月薪 < 2k 或 > 60k 剔除；技能数为 0 的记录**保留**并打标（它本身是有效信息）

3.3 清洗结果与质量校验

清洗后得到 29,999 行 × 27 列。为防止「看起来干净、其实有坑」，我写了 6 项校验逐条核对，全部通过：

校验项	规则	结果
记录唯一性	四字段指纹无重复	`0` 重复
字段完整性	必填字段无空值	`0` 缺失
薪资逻辑	下限 ≤ 上限，且落在 2–60k	`0` 倒挂、`0` 越界
薪资分布	均值 / 中位数是否被极端值拉偏	均值 `16.76k`，中位数 `16.5k`，无明显偏态
技能字段	每个岗位至少有 1 个技能标签	实际范围 `2-6` 项，最少 `2` 项
分类取值	学历 / 经验 / 城市分级取值在枚举内	`3 / 4 / 2` 类，无脏值

校验脚本随报告一并保留，任何人拿到这份 CSV 都能复核上述六项结论。

3.4 一个真实的坑：三个技能字段全是 0

清洗完成后做描述性统计，发现 `skill_机器学习`、`skill_Java`、`skill_BI` 三列合计值都是 0。第一反应是代码写错了，排查顺序是这样的：

1. **怀疑正则写错：**把三列的正则单独跑一遍，用「机器学习」这几个字手工构造一句话，能命中 → 正则没问题。
2. **怀疑全角/大小写：**把中文全角括号、英文大小写、空格变体都试了 → 仍然 0。
3. **回看原文：**随机打印 200 条岗位描述，发现这批 JD 的描述**高度模板化**， 反复出现的只有 Python、SQL、Hadoop、Hive、Spark、Flink、MySQL、数据仓库、Linux、Excel 这十项， 从来没有提到机器学习、Java、BI。

所以这不是代码 bug，而是**数据本身没有这个信息。**处理方式是：不删列（保留可追溯性）， 在图表和报告里显式标注为「采集缺口」，并明确写「不可解读为市场无需求」。

> 这条坑给我的经验
>
> 清洗阶段的「异常」有两种：一种是**数据脏**（格式错、重复、越界），可以靠规则修； 另一种是**数据里根本没有这个信息**（模板化描述、字段缺失），修不出来，只能换数据源或补采集。 分清这两类，才不会把一个采集问题误报成业务结论。

---

4 探索性数据分析

4.0 分析框架：四步递进

四张图不是四种花样，而是一条证据链。顺序刻意从简单到复杂，最后回到「这个结论站不站得住」：

步骤	分析动作	图表	回答
第一步	单变量分布	横向条形图	学什么
第二步	分布形态	柱状图 + 区间高亮	学多宽

步骤	分析动作	图表	回答
第三步	双变量结构	条件概率热力图	怎么组合
第四步	分组对照与相关性	双面板箱线图	可信吗

4.1 各项技能的招聘要求覆盖率

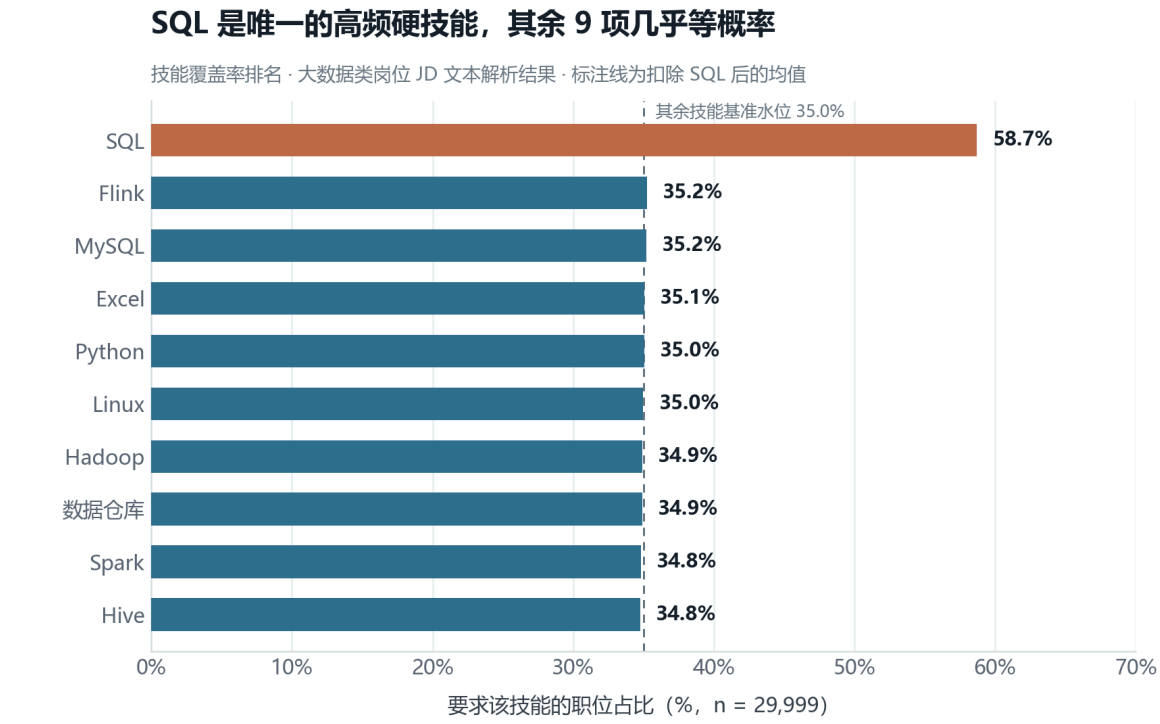


图 1 技能覆盖率排名 | n = 29,999；指标为「该技能字段取值为 1 的岗位占比」；虚线为扣除 SQL 后的均值（35.0%）

读图 SQL 覆盖率 **58.7%**，是第二名 Flink（35.2%）的 **1.67 倍**，断层领先。 后 9 项集中在 34.8%–35.2%，极差只有 **0.4 个百分点**——差异没有业务含义，等于同一条水位线。

图形选择上用了横向条形而不是竖向柱状：技能名是中文长词，横排不需要旋转标签；10 个类别共用一个 0–70% 刻度， 长度可以直接比较；全图只给 SQL 一根强调色，因为一张图只需要让读者记住一个数字。

4.2 单个岗位要求几项技能

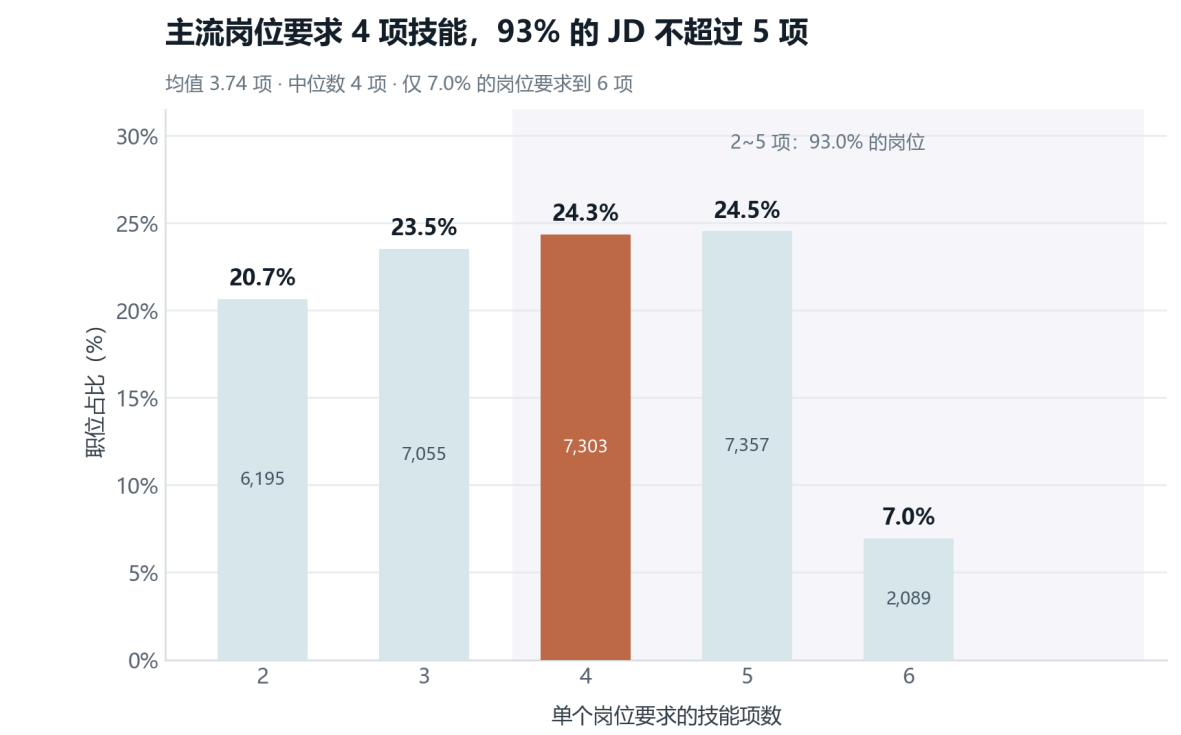


图 2 单岗技能项数分布 | 有效技能 10 项逐行求和；累计口径：≤4 项 68.5%，≤5 项 93.0%

读图 中位数 **4 项**，均值 3.74 项；4 项与 5 项几乎并列（24.3% / 24.5%）； 2–5 项合计覆盖 **93.0%** 的岗位，要求 6 项的仅 **7.0%**。曲线在 5 项之后陡降。

这张图改过一版。初版用双 Y 轴（左职位数、右累计占比），实测累计折线的标注在 5 项处和柱顶数字压在一起。 后来改成单轴：柱子表示占比，累计信息用一条带底色的区间注释表达。信息没丢，可读性反而更好——这也是我对「双轴图」的一次实践判断：**两套刻度同时出现时，读者很难判断该信哪一根。**

4.3 技能之间的绑定关系

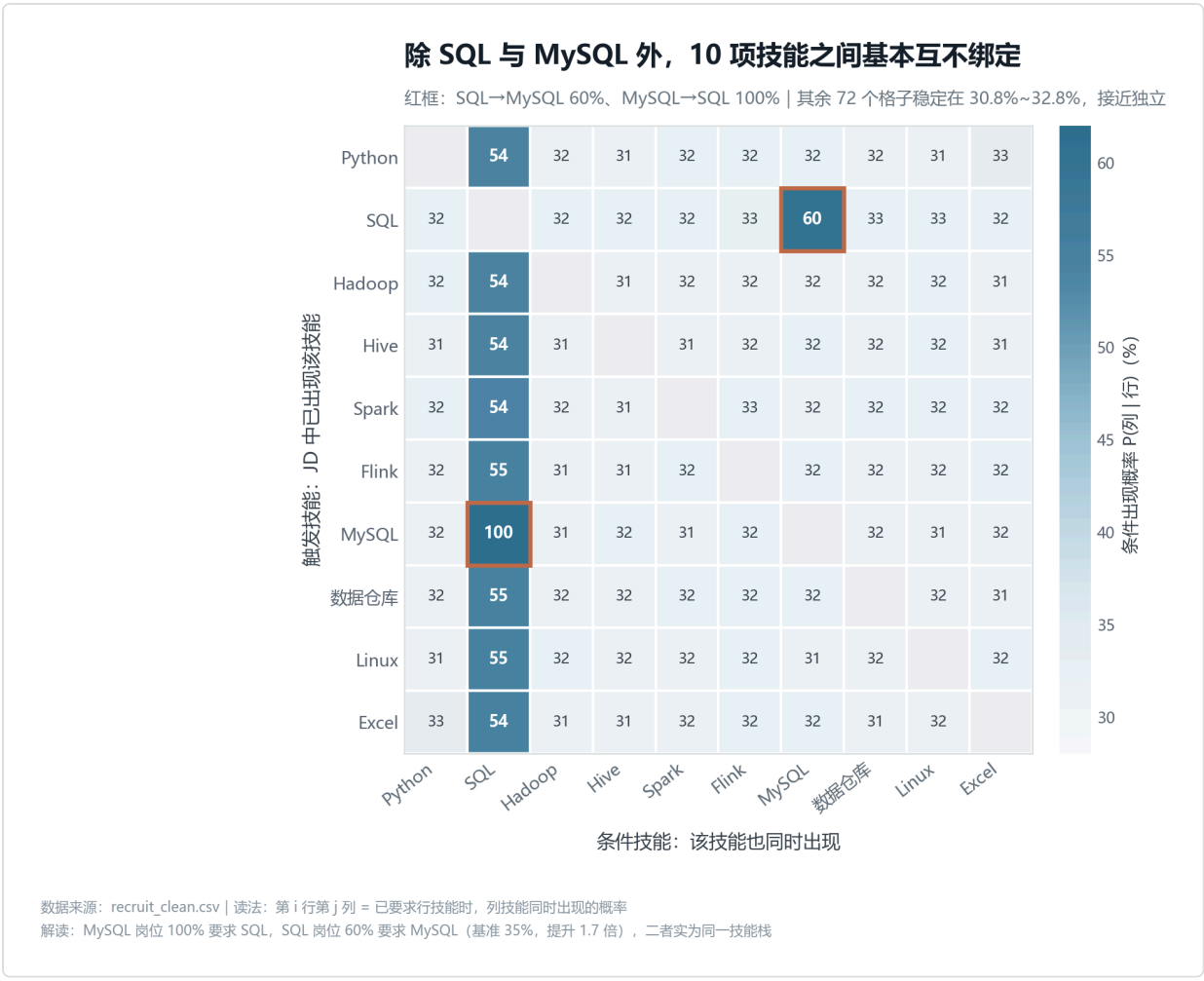


图 3 条件概率矩阵 | 第 i 行第 j 列 = 已要求行技能时，列技能同时出现的概率；对角线已屏蔽

读图 除与 SQL 直接相关的 18 格外，其余 **72 个**非对角格子集中在 **30.8%–32.8%**，与各自基准率（34.8%–35.2%）基本一致，接近统计独立； 唯一强绑定是 **MySQL → SQL 100%**、**SQL → MySQL 60%** （基准 35%，提升 **1.7 倍**）。

这里用条件概率 P(列 | 行) 而不是简单计数，是为了排除基础率的影响：SQL 本身覆盖率高，如果只看计数，它和任何技能的「共现次数」都会很高，看起来像强相关。屏蔽对角线也是同理——10 个 100% 的自相关会压掉所有色阶差异。 另外要说明一点读图陷阱：**SQL 整列偏深（54%–55%）是基础率高的机械结果**，不代表它和每项技能都强相关。

4.4 薪资维度的可信度检验（一个负结果）

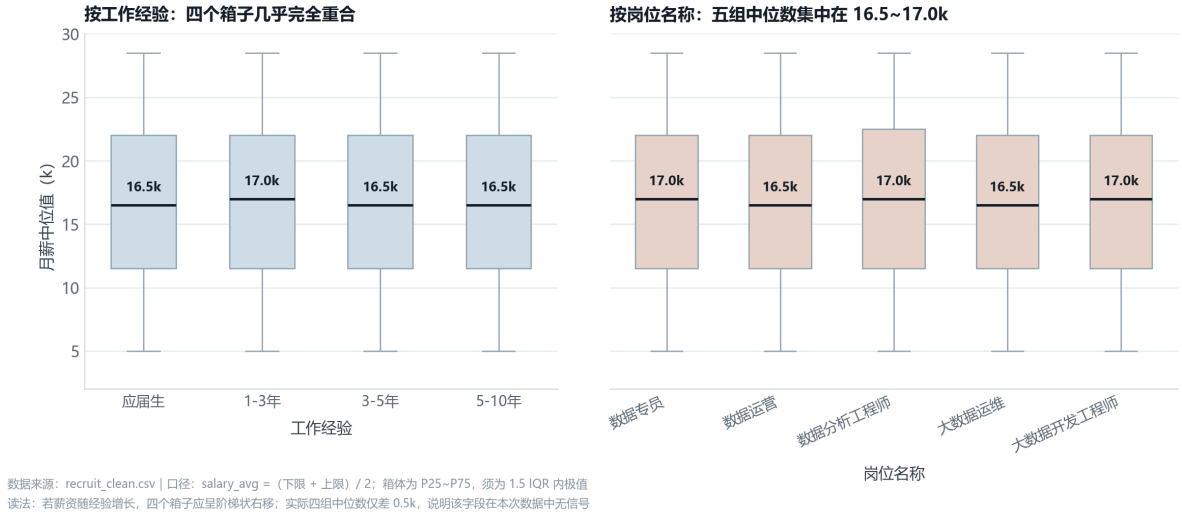


图 4 薪资分组箱线图 | salary_avg =（下限 + 上限）÷ 2；箱体为 P25–P75，须为 1.5 IQR 内极值

读图　四段经验的中位数为 16.5 / 17.0 / 16.5 / 16.5k，只差 **0.5k**，箱体完全重合； 五个岗位中位数 16.5–17.0k，组内标准差 **6.1–6.2k**。

按常识，薪资应该随经验上升、随岗位不同而分层。但四组分布几乎一模一样，这不合常理，于是我做了相关性检验： `salary_avg` 与经验 **r = −0.006**，与学历 **r = −0.002**， 与技能项数 **r ≈ 0**，全部落在噪声范围。

判断与处理

这不是「薪资不涨」，而是**该字段在本数据集中没有信号**（可能与脱敏或生成式填充有关）。 因此它没有进入任何业务结论，只在报告中作为反例保留。硬算一个「3–5 年涨薪 12%」很容易， 面试时被追问一句「这个涨幅怎么来的」就答不上来——**能被追问住的分析，等于没做。**

4.5 四步分析的小结

问题	答案	关键证据	证据强度
学什么	先学 SQL，它与其余 9 项不在一个量级	`58.7% vs 35.0%`	强
学多宽	4 项足够，5 项封顶	中位数 `4`，`93.0% ≤ 5` 项	强
怎么组合	没有成套的「技能套餐」，唯一绑定是 SQL ↔ MySQL	`72` 格 `30.8–32.8%`	强
岗位类别是否有别	5 类岗位的技能要求没有实质差别	同技能极差 `≤ 3.3pp`	中
薪资能否支撑结论	不能	`r = -0.006 / -0.002`	无信号

5 业务结论与项目价值

5.1 四条可以照着做的结论

每条结论都配了三样东西：一句判断、支撑它的证据、以及明天就能执行的动作。

1 把 SQL 放在技能栏第一行，并配一个 MySQL 项目

证据　58.7% 的岗位要求 SQL，是第二名的 1.67 倍；MySQL 岗位 **100%** 也要求 SQL，SQL 岗位有 60% 要求 MySQL（基准 35%）。

动作　技能栏第一行写「**SQL（MySQL）：多表 JOIN、窗口函数、分组聚合**」，不写「熟悉数据库」这种无法追问的话；准备一个 3 张表 + 5 条查询的项目（TopN、连续登录、次日留存、同环比、分组排名），每条查询能说清索引与执行顺序。

2 技能写到 4 项就够，别为「6 项全会」凑数

证据　单岗技能项数中位数 **4 项**，93.0% 的岗位不超过 5 项，要求 6 项的只有 7.0%；5 项之后需求曲线陡降。

动作　用「**1 底座 + 1 硬技能 + 1 方向 + 1 工具**」组配置：SQL（底座）、Python（硬技能）、数据仓库（方向）、Excel / BI（工具）；第 5 项只在目标岗位反复出现时再补。每行技能都要能被追问三层：会用 → 为什么这么用 → 踩过什么坑。

3 别按岗位名称挑方向，这几类岗位要的技能没有区别

证据　同一技能在 5 类岗位的覆盖率极差 **最大 3.3 个百分点**（10 项里 9 项 ≤ 1.7pp）；90 个共现格子里 72 个稳定在 30.8%–32.8%。

动作　做**一份主线简历**（SQL + Python + 数仓项目），只替换项目标题和 3 个业务关键词来对齐 JD 用语：数据运营 →「指标体系 / 活动复盘」，大数据开发 →「离线数仓 / 调度 / 分层」，数据分析 →「归因 / AB 实验」。一份底稿覆盖多类岗位，胜过五份半成品。

4 薪资不要用这份数据谈，改用三个可验证的外部依据

证据　薪资与经验 **r = −0.006**、与学历 r = −0.002；四段经验中位数只差 0.5k，五个岗位中位数 16.5–17.0k，组内标准差 6.1–6.2k。

动作　报告里把薪资标注为「字段无区分度，未采用」，只输出技能结论；谈薪改用三份依据：目标城市同岗位近 3 个月在招 JD 的薪资区间、平台城市分位数据、在手 offer 对标。被问期望薪资时，**给区间不给点值。**

5.2 项目价值

对我自己：把一个模糊的焦虑（「大数据到底要学什么」）变成了可执行的清单。 我不再是「听说要学 SQL」，而是知道 58.7% 的岗位点名要它、并且 MySQL 岗位 100% 也要求它—— 这两句话足够支撑我在简历和面试里把 SQL 讲成主线。

对同学：方法可以复制。四步框架（单变量 → 分布 → 双变量 → 可信度检验）不依赖这份数据， 换一份电商、运营或测试岗的 JD，改一下技能词典就能复用同一套流程和脚本。

方法层面：这个项目最有价值的部分不是「SQL 最高」，而是**先验证字段、再下结论。** 如果我没有做第 4 张图，可能会顺手写一句「3–5 年经验薪资更高」——那是数据里不存在的东西。

工程层面：全流程脚本化，清洗与绘图各自封装成函数，换 CSV 路径即可重跑， 4 张图一条命令输出，每张图自带数据来源标注，便于复核。

5.3 这个项目证明了我具备的四项能力

能力	在项目中的体现
数据处理	把非结构化 JD 文本转成 27 列结构化数据：正则解析薪资、分类编码、词典抽取技能、六项质量校验
分析思维	四步递进的分析框架；用条件概率排除基础率干扰；主动做字段可信度检验并接受负结果
可视化表达	4 张图各承担一个问题；统一配色与标注规范；一次双轴改单轴的设计迭代
工程与沟通	脚本可复跑、结论可追溯；能把统计结果翻译成「明天做什么」的具体动作

---

6 项目总结、局限性与后续可优化方向

6.1 项目总结

这个项目用 29,999 条招聘信息回答了三个求职问题：**SQL 是唯一的高频硬技能（58.7%）；单岗技能要求中位数 4 项、 93% 不超过 5 项；技能之间基本独立，唯一强绑定是 SQL 与 MySQL。**三条结论互相支撑， 最后收敛成一个可执行的组合：SQL + Python + 数据仓库 + 一项工具。

过程中真正的收获在方法上。一是**把文本变成字段**：招聘描述原本只能「读」，做成 0/1 标签后才能比较和排序。 二是**接受负结果**：薪资字段经检验没有区分度，我选择把它写进局限，而不是包装成结论。 三是**约束结论边界**：因为样本是配额抽样，我明确不回答「哪个城市机会更多」，只回答技能结构问题。

如果只用一句话概括：**这份数据的价值不在薪资，而在技能要求的分布**；而分析的价值不在算出多少数字， 在于知道哪些数字不能用。

6.2 局限性

按影响程度从高到低排列，每条都写了「如果重做会怎么办」：

级别	局限	对结论的影响	如果重做
P0	薪资字段无信号（与经验、学历、岗位相关性均接近 0）	薪资类结论全部不可用，已排除出报告	换用真实薪酬数据源（平台薪资报告、城市薪酬分位），或按公司规模、融资阶段分层后再比较
P1	机器学习、Java、BI 三个技能字段覆盖率为 0	无法判断算法岗、BI 岗的技能要求，本报告只适用于数据分析 / 数据开发方向	扩大采集范围到算法、BI、数据产品等岗位，并重建技能词典
P2	样本是按城市与岗位做的配额抽样，各组量级接近	不能回答「哪个城市机会多」「哪类岗位更缺人」这类份额问题	改为按平台自然检索结果全量采集，并记录检索时间与排序方式
P3	技能是 0/1 标签，不含熟练度、年限与项目深度	「了解」与「精通」在数据里没有区别，结论只能到「是否要求」这一层	按「必须 / 优先 / 加分」三级打标，并统计技能在描述中的出现次数作为强度代理

级别	局限	对结论的影响	如果重做
P4	只有单一时间窗口，没有时间维度	看不到技能需求的涨跌趋势，也无法判断结论的时效性	按周采集并保留采集日期，形成时间序列后做趋势与同比

6.3 后续可优化方向

按投入产出比分为三档，短期就能做，中长期需要补技术栈：

周期	方向	具体做法
短期	补齐技能词典	扩充同义词表（如「数仓 / 数据仓库 / DW」、「关系型数据库 / MySQL」），并对「技能数 = 0」的记录单独复盘，确认是模板化描述还是漏匹配
短期	增加交叉维度	把行业、城市层级与技能做交叉，回答「金融行业是否更看重 SQL」「一线与新一线的技能要求是否一致」
中期	从词典升级为文本挖掘	用 jieba 分词 + TF-IDF 找出词典之外的技能词，人工确认后回填词典；同时统计技能词频作为「要求强度」的代理指标
中期	结构化岗位与技能的关系	把「岗位—技能」建成二部图，计算技能的中心度，识别哪些技能是通往多个岗位的枢纽（SQL 预计会排第一，可验证）
长期	做成可更新的看板	采集脚本 + 定时任务 + 数据库（SQL 存储）+ BI 看板，每周自动更新技能覆盖率，把一次性分析变成持续监测
长期	引入真实薪酬校验	接入公开薪酬报告做交叉验证，用外部数据校准本项目缺失的薪资维度，使「技能—薪资」的关系可以真正被分析

> 写在最后
>
> 这是我第一个从采集做到结论的完整项目。它不够完美：数据有缺口，薪资维度失败，样本也不是自然分布。 但每一步我都能说清为什么这么做、以及这么做会带来什么限制——我认为这也是数据分析工作里最该被守住的部分。

数据来源：公开招聘平台岗位信息，清洗后 29,999 条（2026 年 9 月采集）｜原始数据不对外分发，报告仅呈现聚合结果

配套材料：可复跑脚本 `code/recruit_viz.py` ｜交互式图表解读见作品集首页